

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

03 Lexical scope

Modules 1 and 3: binding, environments, and closures

1. Introduction & core concept

Lexical scope determines which declaration a name denotes from the program's textual nesting. A binding associates a name with an entity such as a value, object, or function. Scope is the region in which a binding can be used; lifetime is the period during execution when the corresponding object or state exists. They must not be treated as synonyms.

Languages use lexical scope to support modular reasoning. A function's free names do not ordinarily change meaning merely because a different caller invokes it. Shadowing introduces a distinct binding with the same spelling in a nested region. It can improve locality, but excessive shadowing makes accidental use of the wrong binding harder to detect.

2. Under the hood: the mechanics

A parser records nested blocks and functions. Name analysis classifies declarations and references, and execution connects those references to the relevant environment. Python's function-local classification considers binding operations in the function body; reading a classified local before assignment can raise `UnboundLocalError` [1].

A closure combines callable behavior with access to its surrounding lexical environment. Implementations may retain environment objects or captured values after a creating call returns. Lexical lookup describes meaning, not a mandatory chain of physical hash-table searches. Compilers can resolve or optimize access when observable behavior is preserved.

Learning outcomes

- Distinguish shadowing, rebinding, scope, and lifetime.
- Trace separate closure environments.
- Identify dangerous reference captures in C++.

Examples progress from Python 3 to modern JavaScript to C++17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: local shadowing

```
value = 10

def show():
    value = 20
    print(value)

show()
print(value)
```

In-depth reasoning

Expected result: 20
10

Module execution first binds `value` to 10. The function definition creates a callable without running its body. Calling `show` creates a function activation; the assignment establishes a local binding named `value`, which refers to 20. The `print` inside that activation reads this local binding.

After `show` returns, the module-level `print` still reads the original module binding, so it displays 10. No operation assigned to that binding. Equal spelling does not imply that the variables are the same storage location. The call's control state ends, while any objects retained elsewhere may continue to exist.

If a read of `value` were placed before its assignment inside `show`, Python would still classify `value` as local for that function and the read would fail. It would not automatically fall back to the global 10. A global declaration would deliberately redirect assignment to the module binding; `nonlocal` applies to an existing enclosing function binding.

Compare the language models

Python's ordinary `if/for` blocks do not create the same local scopes as functions. JavaScript `let/const` and C++ local declarations use block scoping, so translating braces or indentation mechanically can change binding behavior.

Checkpoint: Does calling `show` twice change the global value? No; each call creates its own local binding.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: independent closure state

```
function makeCounter() {  
  let count = 0;  
  return function next() {  
    count += 1;  
    return count;  
  };  
}  
  
const first = makeCounter();  
const second = makeCounter();  
console.log(first());  
console.log(first());  
console.log(second());
```

In-depth reasoning

Expected result: 1

2

1

Each call to `makeCounter` creates a distinct count binding. The returned `next` function retains access to that call's environment. `first` and `second` consequently share the same source definition but do not share the same counter state. The first two calls update one binding; the third updates the other.

The environment required by `first` remains available after its creator returns. This does not mean that the creator keeps executing or that its physical stack frame must remain unchanged. The engine preserves the observable state through its closure implementation. Collection is possible only after reachable references no longer require that state.

Changing the outer count declaration to `const` would make `count += 1` fail because the binding cannot be reassigned. It would not create an immutable counter object. Likewise, a caller's own variable named `count` has no effect on the captured binding. Lexical association, rather than caller identity, determines the target.

Compare the language models

Python can express the same counter with an enclosing binding and `nonlocal`. C++ lambdas distinguish copying a captured value from retaining a reference, and those choices impose different lifetime requirements.

Checkpoint: If `second = first` were allowed and performed, would their counters stay independent? No: both would refer to the same returned function and environment.

3. CODE EXAMPLES / C++

Advanced: capture by value versus reference

```
#include <iostream>

int main() {
    int count = 10;
    auto copied = [count]() mutable {
        return ++count;
    };
    auto shared = [&count]() {
        return ++count;
    };
    std::cout << copied() << ' ' << count << '\n';
    std::cout << shared() << ' ' << count << '\n';
    std::cout << copied() << ' ' << count << '\n';
}
```

In-depth reasoning

Expected result: 11 10

11 11

12 11

Creating `copied` stores a copy of the initial integer in the closure object. `mutable` permits its call operator to modify that stored copy. The first invocation increments the copy to 11 while the original count remains 10. Creating `shared` instead gives its body access to the original count through reference capture [3].

Calling `shared` increments the original to 11. The later `copied` call increments its own retained value from 11 to 12. In C++17, the displayed stream operations are sequenced so the count displayed after each call reflects that call's effect. The values make the two state locations visible.

Both closures are used while the original count is alive, so the reference is valid. Returning `shared` from a function whose local count is then destroyed would create a dangling reference; lexical visibility would not extend the integer's lifetime. A value capture is often appropriate when a callback must outlive its creation scope, although `copied` pointer-like values require separate ownership analysis.

Compare the language models

Python and JavaScript closures generally retain access to managed environments. C++ reference capture does not supply that managed lifetime extension. Shared spelling and convenient lambda syntax do not imply identical ownership semantics.

Checkpoint: Why is `mutable` needed only for `copied`? It modifies the closure's captured value; `shared` modifies the separate object referred to.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

Draw bindings as distinct boxes and reference relationships as arrows before predicting output. Identify the declaration selected by each name, then ask whether its associated state is still alive. In Paradigm Diagnostics, the lexical-scope lesson demonstrates local/global shadowing; closures extend the same reasoning to functions that escape their defining call.

Review questions with answer guidance

1. Why can shadowing be harmless yet confusing?

It preserves distinct bindings but hides an outer name, making the intended reference less obvious.

2. Would a function's caller supply its free variables under lexical scope?

Ordinarily no; textual definition context determines those associations, unlike dynamic scoping.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python: execution model and name resolution. [Read source](#)

[2] ECMAScript: execution contexts and environments. [Read source](#)

[3] C++ working draft: lambda captures. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.